29/12/2025

Work done

- **OOP**

Description :

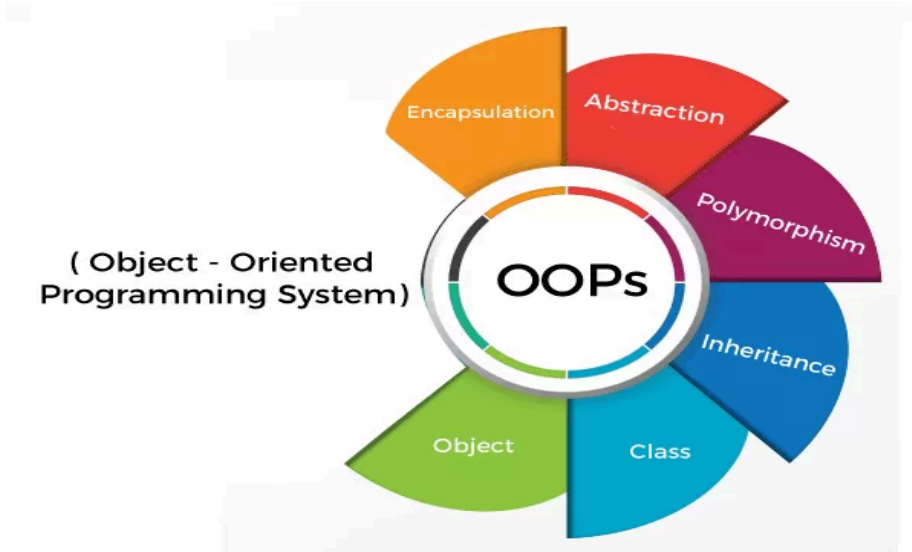
OOP :

➔ OOP means Object Oriented Programming
➔ Python is an object oriented programming language .

Advantages of OOP

1. It gives a clear structure for the program.
2. make it easier to maintain, reuse, and debug the code.
3. Help to keep your code DRY(Don't repeat yourself - The DRY principle means you should avoid writing code more than once. Repeated code should write in functions or class and reuse it.)
4. Provides reusability feature
5. Provides modular structure (modules - small parts of code)

OOP Concepts :



1. Class

- ➔ Class is a collection of objects.
- ➔ Class is a blueprint for creating objects.
- ➔ Class defines a set of attributes and methods.
- ➔ For creating a class , we use the **class** keyword.
- ➔ Class attributes = variables of class.
- ➔ Class methods = functions in class.
- ➔ Class attributes can be accessed using dot(.) operator. Ex., dog.name , here "dog" is class name and "name" is an attribute of dog.
- ➔ Behaviour = represent by method
- ➔ Properties = represent by variable or attribute

Class → House blueprint

`__init__()` → Construction process

Object → Actual house

Creating a class :

Syntax:

```
class class_name:
        Block of code
```

Example:

```
        class car :
                brand  = " BMW "  # class attribute

                def __init__(self , model , color):
                        self.model = model  # instance Attribute
                        self.color = color   # instance attribute
```

Creating object :

Syntax :

```
Object_name = class_name()
```

Example:

```
        C1 = car()
```

Accessing attributes :

Syntax:
Class_name.attribute_name
Class_name.instance_name

Example:

```
class car :
        brand  = “ BMW “  # class attribute

        def __init__(self , model , color):
            self.model = model  # instance Attribute
            self.color = color   # instance attribute

        def show(self):
            print(“Hello BMW owners !!!”)

C1 = car(“M8” , “Black”)

C1.show()
print(C1.brand)  # BMW
print(C1.model)  # M8
print(C1.color)  # Black
```

self :

- ➔ self is not a keyword , it is a convention.
- ➔ Self refers to the current object(instance).
- ➔ It is used to access methods and variables of the same object inside the class.
- ➔ Used to access = instance variable + instance method

Example:

```
class car :
    def show(self, name):
        self.name = name
        print("I have " + self.name + " car. ")

C1 = car()
C1.show("BMW")

C2 = car()
C2.show("PORSCHE")

C3 = car()
```

```
C3.show("THAR")
```

```
I have BMW car.
I have PORSCHE car.
I have THAR car.
```

Explanation :

C1 = car()
C1.show(“BMW”) # I have a BMW car.

Internally it means :
car.show(C1 , “BMW”)

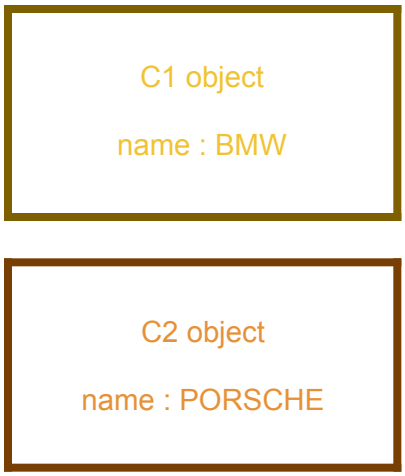
So here self = C1

C2 = car()
C2.show(“PORSCHE”) # I have a Porsche.

Internally it will be like this :
car.show(C2 , “PORsCHE”)

So here self = C2

In memory :



C3 object

name : THAR

Whenever C1 object calls then it will print C1 instance value and this same process will happen with all objects of car

Due to self :

- Avoid confusion
- Data is separate

```
class student :
    name = "unknown" # class attribute

    def __init__(self , name):
        self.name = name # instance attribute

    def show(self):
            print(self.name)

s1 = student("Sayma")

print(s1.name) # priority : instance attribute > class attribute
print(student.name) # calling class attribute if both have name
s1.show()
```

Output :

Sayma
unknown
Sayma

Here we call name directly but it prints instance attribute value because

- Instance attribute priority > class attribute priority
- After creation of an object , python checks the first object (instance). If python found attribute there then it stop searching and print instance value of object

Attribute Lookup Order:

1 Check inside **s1 object (instance)**
2 If NOT found → check **Student class**
3 If NOT found → check parent classes

- **In this situation you want to print a class attribute then its syntax will be - class_name.attribute_name .**
- **We can modify the data :**

  object_name.attribute_name = new_value

- **We can delete attribute :**

  del object_name.attribute_name

Python __init__() Method

- ➔ __init__ is a constructor called automatically when an object is created.
- ➔ __init__ does not return any value .
- ➔ It is used to initialize(set) the data members(attributes and methods) of class when object is created
- ➔ It is called once per object .
- ➔ It must have self as a first parameter .

Syntax:
class ClassName:
  def __init__(self):
    # initialization code

constructor without parameter

```
class person :
    def __init__(self):
        self.name = "unknown"

    def show(self) :
        print(self.name)

p1 = person()
```

```
p1.show()
```

Output:
unknown

constructor with parameter

```
class flower :

    def __init__(self , name , color):

        self.name = name

        self.color = color



    def display(self) :

        print("I want " + self.name + " with " + self.color +" color")

f1 = flower("rose" , "white")

f1.display()
```

Output :

I want rose with white color

Default value in __init__

```
class interns:

    def __init__(self , name, company ="Mitu Skillologies", interval =
"6"):

        self.name = name

        self.company = company
```

```
        self.interval = interval


    def info(self):

        print("I'm "+self.name)

        print("Currently, I'm working as an intern in "+self.company+

        " for "+self.interval+" months duration.")


i1 = interns("Sayma Kazi")

i1.info()
```

Output:

I'm Sayma Kazi

Currently, I'm working as an intern in Mitu Skillologies for 6 months duration